

Module 7: Lesson 4

Momentum Timing with MLPs



Outline

- ▶ Constructing an MLP model
- ▶ MLPs for factor timing
- ▶ Backtesting the 'Mom timing' strategy

MLP model

This lesson will revisit a lot of the fundamental finance concepts of Lesson 2, where we deployed linear regression to develop a trading strategy based on momentum factor timing.

Now, however, we are going to build our first deep learning model using MLPs. In this case, we will choose a model with 4 layers:

- ▶ 3 Hidden layers with 50 neurons/units, 30 units, and 10 units, respectively
- ▶ Single-unit output layer

Model: "sequential"		
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	800
dense_1 (Dense)	(None, 30)	1530
dense_2 (Dense)	(None, 10)	310
dense_3 (Dense)	(None, 1)	11
Total params: 2,651		
Trainable params: 2,651		
Non-trainable params: 0		

Training the MLP model

Training the MLP model is going to be a little bit more complicated. We will have to make choices on:

- ▶ "Usual suspects": optimizer (Adam), learning rate (10^{-5}), loss function (MAE)
- ▶ Activation functions ('ReLU' for all units)
- ▶ EarlyStopping: stopping criteria, patience,...

```
act_fun = 'relu' # Activation function
hp_units = 50    # Units in the first hidden layer
hp_units_2 = 30  # Units in the second hidden layer
hp_units_3 = 10  # Units in the third hidden layer

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units = hp_units, activation=act_fun))
model.add(tf.keras.layers.Dense(units = hp_units_2, activation=act_fun))
model.add(tf.keras.layers.Dense(units = hp_units_3, activation=act_fun))
model.add(tf.keras.layers.Dense(1))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)
model.compile(optimizer = adam, loss= 'mean_absolute_error')
es = tf.keras.callbacks.EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=10, restore_best_weights=True)
model.fit(X_train, y_train, validation_split=0.2, epochs=100, batch_size=32, verbose=2, callbacks=[es])
```

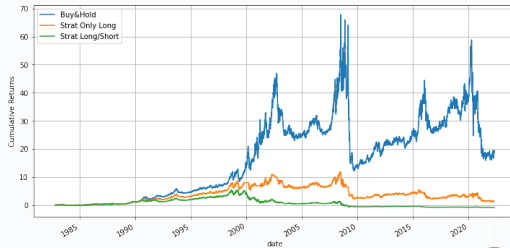
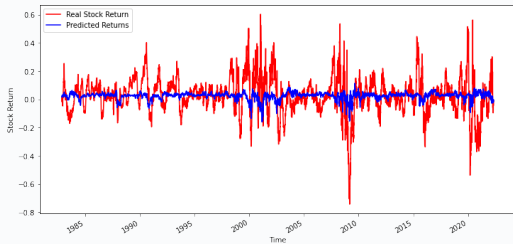
Performance of MLP model

As in previous cases, we will look at the performance of the model for predictive purposes by analyzing:

- Out-of-sample predictive power: E.g., $R^2_{OS} = 2.19\%$

Then, we can assess the viability of the trading strategy:

- Evaluating predicted returns vs. real ones in test sample (Backtest):



Summary of Lesson 4

In Lesson 4, we learned how to:

- ▶ Build an MLP model in Tensorflow-Keras
- ▶ Train the MLP model using features like EarlyStopping
- ▶ Backtest a trading strategy based on MLPs

⇒ **References for this Lesson:**

Chapter 5, Section 5.5.3. of [Zhang, Aston, et al. *Dive into Deep Learning*. 2021.](#)

TO DO NEXT: Go to this lesson's associated Jupyter Notebook. There, you will work on your first neural network model using MLPs to develop a momentum timing trading strategy.

This has been all for the Machine Learning course. We will dig much deeper into the deep learning models introduced in this last module in the next Deep Learning for Finance course. See you there!